

PROJEKT z przedmiotu Przetwarzanie Rozproszone 2024/25

Temat: **Współdzielona Tablica Interaktywna**

Grupa Projektowa:

1. Amarasekara Amile
2. Borkowska Martyna
3. Doga Kacper
4. Glaza Karolina
5. Pawłowska Agnieszka
6. Wołoszyn Dawid

1. Cel projektu

Celem projektu było stworzenie aplikacji umożliwiającej wielu użytkownikom jednocześnie rysowanie na wspólnym obszarze roboczym w czasie rzeczywistym. System został zaprojektowany w architekturze klient-serwer i uwzględnia przesyłanie zmian w zawartości obszaru, synchronizację jego stanu oraz zarządzanie pozycjami kursorów poszczególnych użytkowników.

2. Opis ogólnej architektury

System składa się z następujących komponentów:

- **Serwer:** obsługuje wielu klientów, gromadzi zmiany i informacje dotyczące kursorów i rozsyła je do wszystkich podłączonych użytkowników.
- **Klient:** umożliwia rysowanie, wysyłanie zmian do serwera oraz aktualizację lokalnego obszaru roboczego na podstawie przychodzących danych.
- **Bufory i modele danych:** zarządzają zmianami obszaru roboczego i metodą przekazywania informacji pomiędzy serwerem a klientem.

3. Model działania

1. Nawiązanie połączenia klienta z serwerem

- Klient tworzy połączenie typu TCP/IP z serwerem.
- Po stronie serwera *ServerSocket* akceptuje połączenie i przypisuje każdemu klientowi osobny wątek, odpowiedzialny za jego obsługę.

2. Tworzenie i serializacja zmiany rysunku

- Gdy użytkownik rysuje linię, kolejne punkty są dodawane do bufora (*ChangeBuffer*) w postaci jednostek *CanvasChange*.
- W regularnych odstępach czasu (lub po zapełnieniu bufora), *ChangeBuffer* buduje paczkę *CanvasChangeCompressed*, zawierającą:
 - typ zmiany (*Draw/Erase*),
 - współrzędne i rozmiar zmienionego obszaru,
 - kolor,
 - maskę (ograniczającą rozmiar przesłanych danych).
- Obiekt ten jest serializowany i wysyłany do serwera.

3. Obsługa zmian na serwerze

- Serwer odbiera dane od klienta w dedykowanym wątku.
- Po zdeserializowaniu obiektu *CanvasChangeCompressed*, serwer natychmiast rozsyła go do wszystkich pozostałych klientów (z wyjątkiem nadawcy).

4. Aktualizacja płótna u klienta

- Każdy klient odczytuje dane za pomocą *BufferReader*.
- Po odebraniu nowego *CanvasChangeCompressed*, klient dekompresuje go (lub bezpośrednio przekształca na *CanvasChange*) i nakłada zmiany na lokalnym *CanvasBuffer*.
- Zaktualizowany stan płótna jest odzwierciedlany graficznie przez komponent Swing, który odświeża widok po wywołaniu metody *repaint()*.

4. Synchronizacja:

1. Synchronizacja zmian rysunkowych

Proces synchronizacji zmian rysunku przebiega według następującego schematu:

1. **Lokalne nanoszenie zmian** – po każdym działaniu użytkownika zmiany są rejestrowane jako obiekty *CanvasChange* i od razu nakładane na lokalny *CanvasBuffer*.
2. **Buforowanie i kompresja** – zmiany są tymczasowo przechowywane w buforze, a co około 40 ms pobierane i grupowane według typu (*DRAW/ERASE*) i koloru. Powstałe grupy są przekształcane do zoptymalizowanych paczek *CanvasChangeCompressed* przy użyciu metody *compress()*.
3. **Wysyłanie do serwera** – paczki są serializowane do formatu JSON (przy pomocy biblioteki GSON) i przesyłane do serwera przez socket TCP.
4. **Rozgłaszanie zmian przez serwer** – po odebraniu paczki serwer rozsyła ją do wszystkich innych klientów.
5. **Aktualizacja płótna u odbiorców** – klient odbierający paczkę dekompresuje ją (metodą *decompress()*), odzyskuje oryginalne zmiany i stosuje je do swojego lokalnego *CanvasBuffer*.

Dzięki temu każdy klient widzi te same zmiany w niemal rzeczywistym czasie, przy czym lokalny użytkownik widzi swoje działania natychmiast, a inni z niewielkim opóźnieniem.

5. Przesył danych:

W projekcie dane są przesyłane w formacie JSON, ponieważ jest to uniwersalny format wymiany danych, który dobrze współpracuje z biblioteką GSON używaną w aplikacji. Dzięki temu możliwa jest szybka i bezpośrednia serializacja i deserializacja obiektów, co upraszcza komunikację między klientem a serwerem.

Z kolei przesyłanie zmian w formie paczek – a nie pojedynczych punktów – wynika z potrzeby optymalizacji. Podczas rysowania powstaje wiele zmian w krótkim czasie, dlatego są one chwilowo buforowane i grupowane, a następnie kompresowane do postaci jednej paczki zawierającej dane o wspólnym obszarze oraz masce bitowej. Takie podejście znacząco ogranicza liczbę wysyłanych wiadomości i zmniejsza rozmiar przesyłanych danych, co poprawia wydajność aplikacji i zmniejsza opóźnienia w synchronizacji między użytkownikami.

6. Ważne klasy i metody:

1. Klasa *Client*:

Klasa *Client* stanowi główny komponent aplikacji po stronie użytkownika. Jej zadaniem jest obsługa interfejsu graficznego, reagowanie na działania użytkownika (np. rysowanie, wybór koloru, zmiana grubości pędzla), przesyłanie zmian do serwera oraz odbieranie i nakładanie zmian przesłanych przez innych użytkowników. Dodatkowo odpowiada za synchronizację pozycji kursora oraz zapis tablicy do pliku graficznego.

Główne funkcje klasy *Client*:

1. **Tworzenie połączenia z serwerem** – klient łączy się z serwerem przez socket TCP i wykonuje początkowy handshake, odbierając unikalny identyfikator *userId*.
2. **Interfejs graficzny** – zbudowany przy użyciu *JFrame* i *JPanel*, zawiera:
 - obszar roboczy (tablica),
 - pasek narzędzi z wyborem kolorów, suwakiem grubości pędzla oraz przyciskiem do zapisu obrazu.
3. **Rysowanie** – reakcja na zdarzenia myszy (*mousePressed*, *mouseDragged*) powoduje lokalne nanoszenie zmian na obiekt *CanvasBuffer* w postaci *CanvasChange*.
4. **Synchronizacja kursora** – klient na bieżąco przesyła swoją pozycję kursora oraz odbiera pozycje innych użytkowników, które są wizualizowane w widoku.
5. **Wysyłanie zmian** – w osobnym wątku (co ~40 ms) klient:
 - pobiera lokalne zmiany z *CanvasBuffer*,
 - kompresuje je do *CanvasChangeCompressed*,
 - przesyła do serwera jako JSON.
6. **Odbieranie zmian** – drugi wątek klienta nasłuchuje na dane z serwera:
 - jeśli odbierze pozycję kursora – aktualizuje widok,
 - jeśli odbierze zmiany (*CanvasChange* lub *CanvasChangeCompressed*) – dekompresuje i stosuje je do płótna.

Najważniejsze metody:

- **main()** – centralna metoda uruchamiająca aplikację, budująca GUI i tworząca wątki *sender* i *receiver*.
- **draw(MouseEvent e)** – obsługuje rysowanie poprzez generowanie i dodawanie *CanvasChange* do *CanvasBuffer*.
- **updateCursor(MouseEvent e)** – aktualizuje pozycję lokalnego kursora.

Uwagi techniczne:

- Do komunikacji sieciowej użyto *Socket*, *BufferedReader* i *PrintWriter* z automatycznym flushowaniem.
- Do kodowania i dekodowania danych wykorzystano bibliotekę **Gson**, umożliwiającą elastyczne operowanie na JSON-ach.

2. Klasa **Server**:

Klasa *Server* realizuje centralny komponent systemu klient-serwer, odpowiedzialny za:

- przyjmowanie połączeń od klientów,
- rozsyłanie informacji o zmianach rysunkowych i pozycjach kursorów,
- buforowanie i kompresję danych,
- utrzymywanie spójnego stanu płótna w pamięci serwera.

Aplikacja nasłuchuje na ustalonym porcie TCP i obsługuje wielu klientów jednocześnie, przydzielając każdemu z nich osobny wątek.

Główne funkcje klasy **Server**:

1. Obsługa połączeń klientów

Serwer nasłuchuje połączeń przy użyciu *ServerSocket*. Po odebraniu połączenia inicjalizowany jest nowy wątek, który wykonuje:

- handshake z klientem (przydzielenie unikalnego identyfikatora *userId*),
- przesłanie stanu początkowego płótna w postaci skompresowanych zmian (*CanvasChangeCompressed[]*),
- rozpoczęcie dwukierunkowej komunikacji tekstowej opartej na JSON.

2. Przechowywanie stanu płótna

Stan całego płótna przechowywany jest w mapie *ConcurrentHashMap<Point, String> board*, gdzie *Point* oznacza współrzędne pikseli, a *String* to wartość koloru w formacie HEX. Serwer modyfikuje tę mapę na podstawie odebranych zmian typu *"DRAW"* i *"ERASE"*.

3. Buforowanie i rozsyłanie zmian

Zmiany są dodawane do bufora (*ChangeBuffer*) i nie są rozsyłane natychmiast. W osobnym wątku "flusher" serwer co 20 ms sprawdza, czy bufor zawiera nowe dane i, jeśli tak, kompresuje je do *CanvasChangeCompressed[]*, serializuje do JSON i rozsyła do wszystkich podłączonych klientów za pomocą metody *broadcast(String msg)*.

4. Synchronizacja kursorów

Serwer odbiera od klientów dane o położeniu kursorów (*CursorPosition*) i rozsyła je w czasie rzeczywistym do pozostałych uczestników sesji.

5. Walidacja danych

Przed nałożeniem zmiany na tablicę serwer sprawdza, czy współrzędne mieszczą się w dozwolonym zakresie (*isValid()*), co zapobiega błędom przesyłania lub manipulacji.

Najważniejsze metody:

- ***main()*** – uruchamia główną pętlę serwera oraz wątek do okresowego rozsyłania buforowanych zmian.
- ***handleClient(ServerSocket server)*** – obsługuje pełen cykl życia klienta: połączenie, handshake, odbiór danych, dekodowanie zmian i synchronizację.
- ***apply(Collection<CanvasChange>)*** – aktualizuje lokalny stan tablicy oraz dodaje zmiany do bufora wysyłki.
- ***broadcast(String msg)*** – rozsyła dane JSON do wszystkich aktywnych klientów.
- ***fullBoardAsChanges()*** – konwertuje cały stan tablicy na listę zmian *CanvasChange* (np. w celu przesłania jej nowemu klientowi).

Uwagi techniczne:

- Komunikacja z klientami odbywa się tekstowo, w formacie JSON.

3. Klasa *CanvasBuffer*:

Klasa *CanvasBuffer* pełni kluczową rolę po stronie klienta jako lokalny bufor stanu tablicy oraz tymczasowych zmian rysunkowych. Odpowiada zarówno za przechowywanie bieżącej zawartości płótna (kolorów wszystkich aktywnych punktów), jak i za zbieranie zmian oczekujących na przesłanie do serwera.

Główne funkcje klasy:

1. Przechowywanie stanu płótna

- Główne pole *canvasState* to mapa *ConcurrentHashMap<Point, String>* odwzorowująca aktualny stan płótna. Każdy wpis zawiera pozycję punktu i przypisany kolor w formacie HEX.
- Jest to stan odpowiadający aktualnemu widokowi rysunku u klienta, odzwierciedlający to, co użytkownik widzi w GUI.

2. Buforowanie zmian do wysyłki

- Pole *pendingChanges* to lista (*CopyOnWriteArrayList*) przechowująca zmiany (*CanvasChange*) dodane przez użytkownika, które nie zostały jeszcze wysłane do serwera..
- Lista ta pełni funkcję kolejki wyjściowej, z której zmiany są okresowo pobierane i wysyłane do serwera po kompresji.

Najważniejsze metody:

- ***applyChange(CanvasChange change)*** – nakłada zmianę na płótno: dodaje lub usuwa punkt z *canvasState*, a następnie zapisuje zmianę w *pendingChanges*, niezależnie od typu operacji (*DRAW / ERASE*).
- ***getPendingChanges()*** – zwraca kopię listy oczekujących zmian i jednocześnie ją czyści. Dzięki temu klient może cyklicznie pobierać nowe zmiany do wysyłki.

Uwagi techniczne:

- Bufor działa na zasadzie jednokierunkowego przepływu danych: rysowanie → zapis do bufora → odczyt i przesłanie do serwera.
- W przypadku odbioru zmian z serwera (np. od innego klienta), *applyChange()* jest również używana do nałożenia ich na lokalny widok.

4. Klasa *CanvasChange*:

Klasa *CanvasChange* reprezentuje pojedynczą zmianę na płótnie – jest to podstawowa jednostka rysowania używana przez klienta oraz serwer. Każdy obiekt tej klasy zawiera informacje o typie zmiany, lokalizacji punktu, kolorze i rozmiarze narzędzia.

Główne funkcje klasy:

1. Podstawa do dalszego przetwarzania

- Jest wykorzystywana zarówno do buforowania lokalnych zmian (*CanvasBuffer*), jak i do ich przesyłania oraz dekodowania w ramach komunikacji klient-serwer.
- Stanowi podstawę do rekonstrukcji płótna (np. przy wczytywaniu pełnego stanu).

Uwagi techniczne:

- Kolor przechowywany jest jako string w formacie HEX (zgodnie z wymaganiami GSON-a).

5. Klasa *CanvasChangeCompressed*:

Klasa *CanvasChangeCompressed* służy do reprezentacji zoptymalizowanej paczki zmian na płótnie. Jej celem jest zredukowanie rozmiaru przesyłanych danych poprzez kompresję grupy pojedynczych zmian (*CanvasChange*) do zwartej postaci.

Główne funkcje klasy:

1. Kompaktowa reprezentacja obszaru zmian

- Obiekt klasy *CanvasChangeCompressed* opisuje prostokątny obszar płótna, w którym zaszły zmiany, wraz z informacją o typie operacji (*DRAW / ERASE*), kolorze, wymiarach i położeniu

2. Maska bitowa dla oszczędności danych

- Najważniejszym elementem klasy jest pole mask – łańcuch znaków zawierający ciąg '0' i '1' w porządku wierszowym, wskazujący, które piksele w danym prostokącie faktycznie zostały zmodyfikowane. Dzięki temu zamiast przysyłać pełne dane każdego punktu, wystarcza jeden bit informacji na piksel.

Najważniejsze metody:

- ***decompress()*** - Odtwarza listę pojedynczych zmian (*CanvasChange*) z zakodowanej paczki, analizując maskę bitową. Jest wykorzystywana po stronie klienta i serwera do rekonstrukcji oryginalnych zmian na płótnie.
- ***compress(List<CanvasChange> changes)*** - Metoda statyczna grupująca listę zmian według typu i koloru, a następnie kompresująca każdą z grup do osobnych obiektów *CanvasChangeCompressed*. Dla każdej grupy obliczany jest minimalny prostokąt obejmujący wszystkie punkty oraz tworzona maska bitowa.

Uwagi techniczne:

- Grupowanie zmian w *compress()* odbywa się na podstawie kombinacji *type|color*.
- Wydajność tej reprezentacji jest szczególnie korzystna przy przesyłaniu dużych, spójnych obszarów rysunku – zamiast setek obiektów *CanvasChange*, przesyłany jest jeden zwarty blok danych.

6. Klasa *ChangeBuffer*:

Klasa *ChangeBuffer* odpowiada za buforowanie i grupowanie zmian po stronie serwera przed ich rozesłaniem do klientów. Umożliwia przesyłanie zmian w zoptymalizowanych paczkach w odpowiednich odstępach czasu.

Główne funkcje klasy:

1. Buforowanie zmian *CanvasChange*

- Zgromadzone w czasie rzeczywistym zmiany (np. rysowanie pojedynczych punktów przez użytkowników) są tymczasowo przechowywane w liście *changes*. Zmiany te są dodawane w sposób bezpieczny dla wątków przy użyciu blokady *ReentrantLock*.

2. Grupowanie i kompresja zmian

- W momencie spełnienia warunków (liczba zmian lub czas oczekiwania), bufor zwraca spakowaną listę zmian w postaci obiektów *CanvasChangeCompressed*, umożliwiając ich efektywne rozesłanie do wszystkich klientów.

Najważniejsze metody:

- ***addChange(CanvasChange change)*** - Dodaje pojedynczą zmianę do bufora. Metoda ta jest bezpieczna w środowisku wielowątkowym dzięki synchronizacji za pomocą lock.
- ***getCompressedChanges()*** - Zwraca wszystkie dotychczas zebrane zmiany w postaci skompresowanej (*List<CanvasChangeCompressed>*), a następnie czyści bufor i aktualizuje znacznik czasu.
- ***shouldFlush()*** - Określa, czy należy już rozesłać zgromadzone zmiany. Rozsyłanie następuje, gdy minęło więcej niż *MAX_BATCH_AGE_MS* milisekund od ostatniego wysłania lub bufor zawiera więcej niż *MAX_BATCH_SIZE* zmian.

Uwagi techniczne:

- Parametry *MAX_BATCH_SIZE* i *MAX_BATCH_AGE_MS* definiują kompromis między responsywnością a efektywnością transmisji.

7. Klasy *Point* i *CursorPosition*:

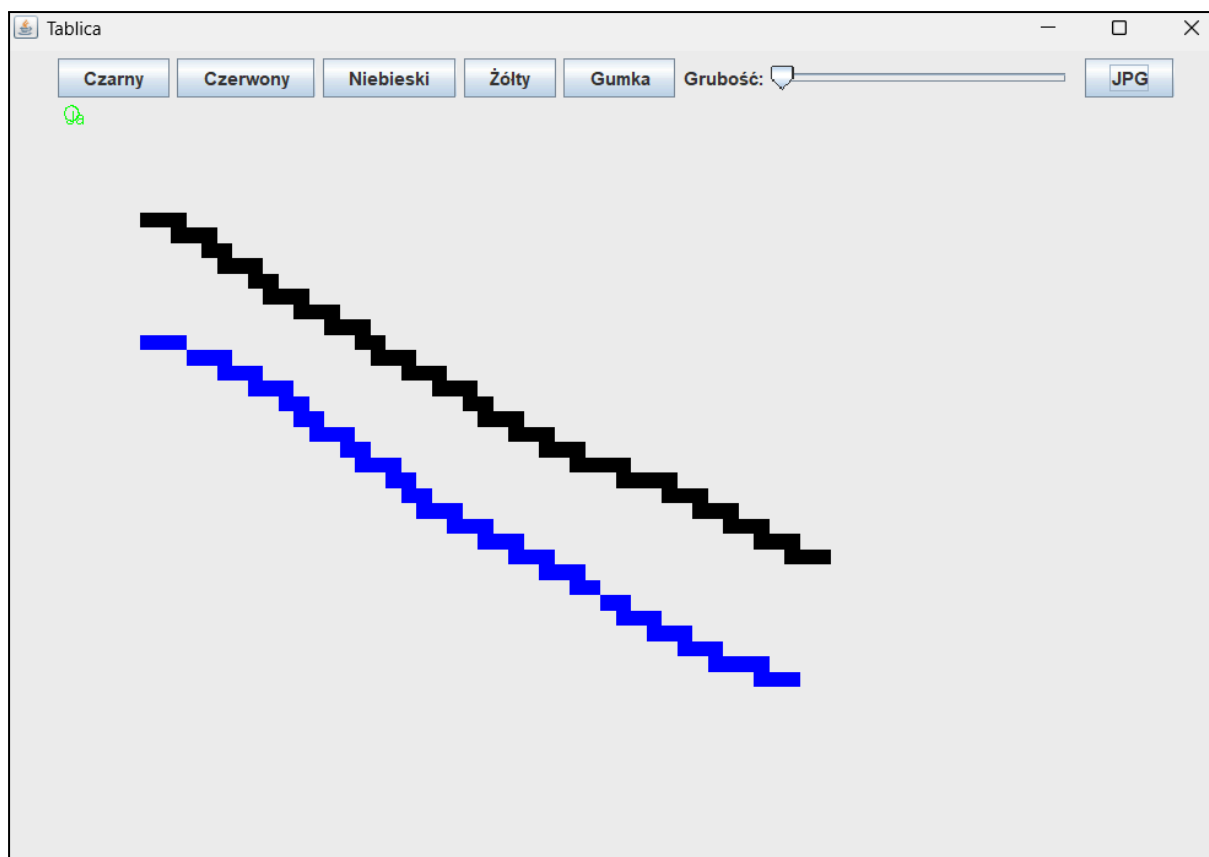
Klasy *Point* i *CursorPosition* są klasami pomocniczymi do metod obsługujących piksele oraz kursory użytkowników.

7. Interfejs użytkownika:

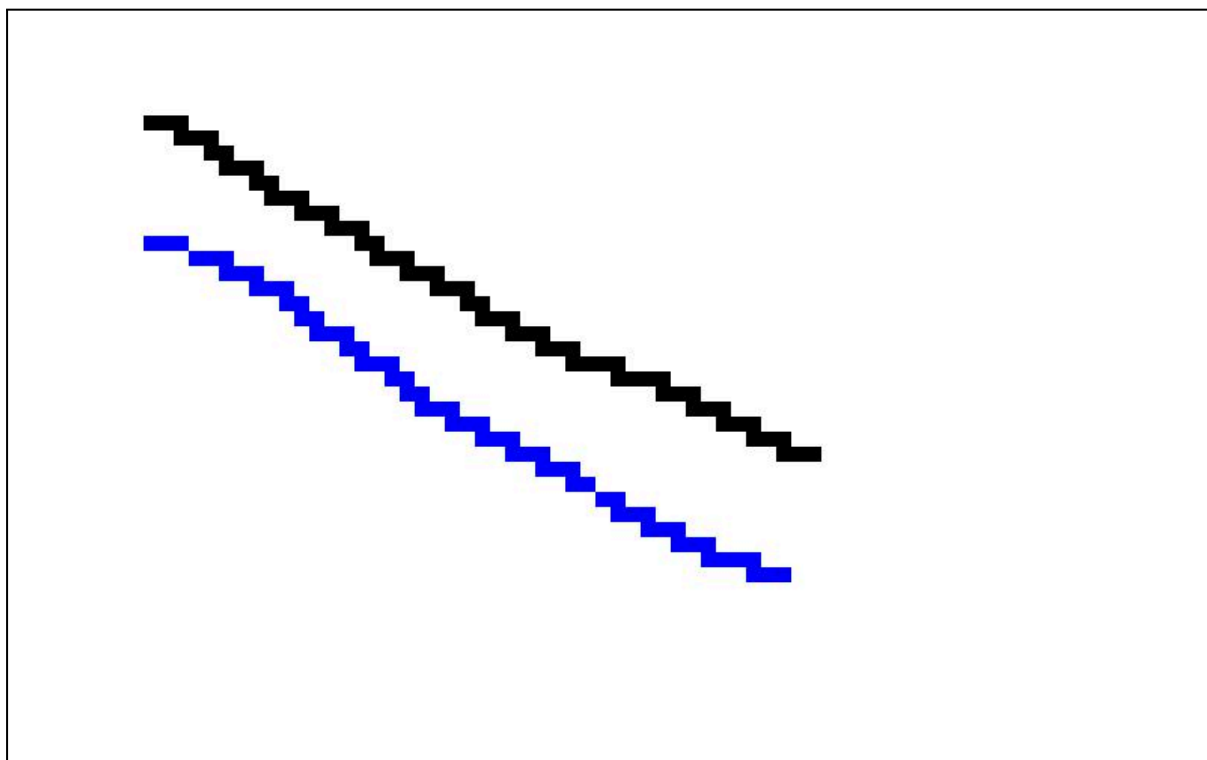
Aplikacja umożliwia użytkownikowi modyfikację narzędzia do rysowania. Dostępne są:

- 4 kolory rysika: czarny, czerwony, niebieski, żółty
- Gumka
- Możliwość zmiany grubości zarówno rysika jak i gumki w skali od 1 do 5 pikseli

Ponadto istnieje możliwość zapisania aktualnego stanu tablicy do pliku *tablica.jpg*

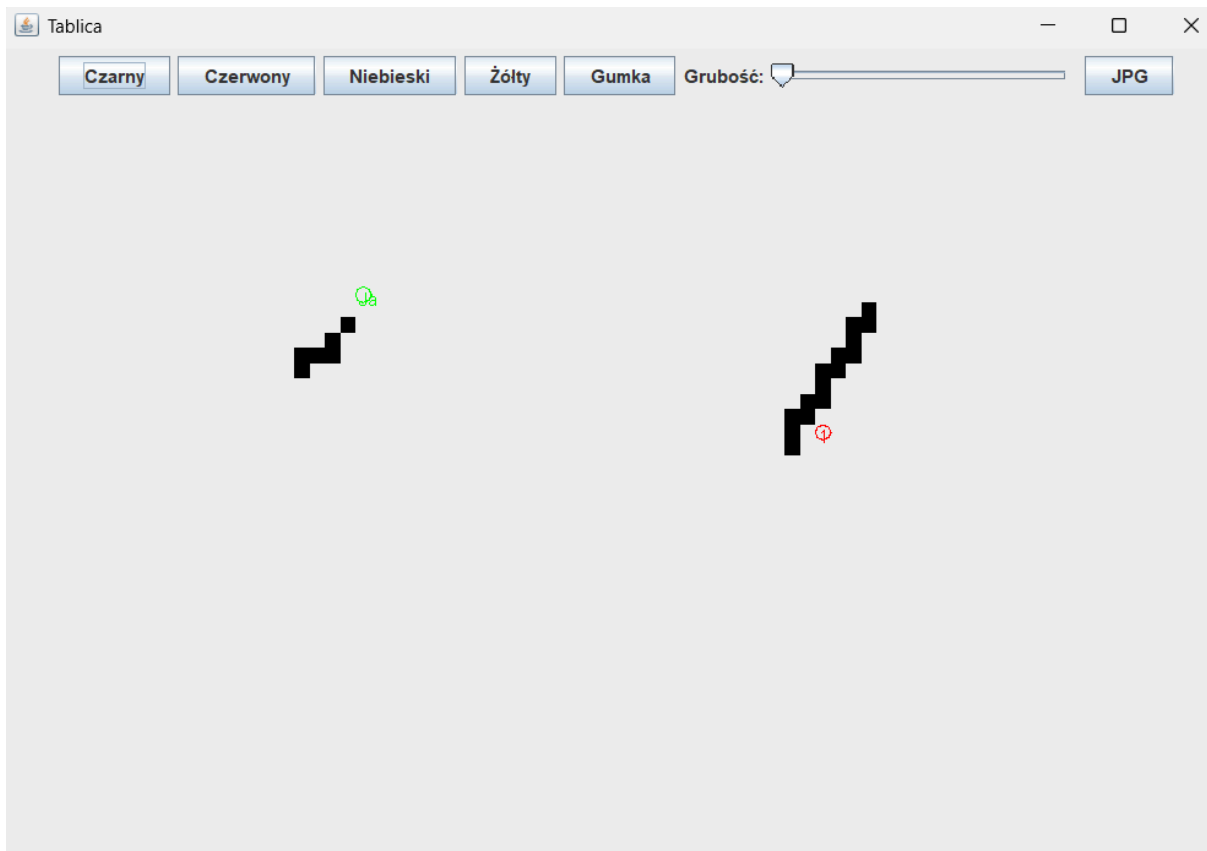


Okno widoczne dla użytkownika



Obszar roboczy zapisany do pliku tablica.jpg

Na obszarze roboczym widoczne są kursory własny - oznaczony zielonym kolorem, jak również kursory innych użytkowników aktualnie edytujących tablicę - oznaczone czerwonym kolorem oraz numerem ID użytkownika, do którego przynależy dany kursor.



Kursory własny i innego użytkownika widoczne na tablicy

8. Testowanie

W ramach testów przeprowadzone zostały najróżniejsze scenariusze, od bazowych takich jak podgląd stanu przez wielu klientów, wspólną edycję różnych sekcji płótna, po edycje i czyszczenie tego samego obszaru.

Synchronizacja dla wielu klientów działa bez zarzutów, opóźnienie nakładania zmian jest na tyle małe, że umożliwia swobodną współpracę w ramach płótna. Edycja różnych obszarów nie wykazuje problemów w swojej funkcjonalności. Czyszczenie płótna działa płynnie, również w przypadku czyszczenia przez kilku użytkowników tego samego obszaru.

Scenariuszem testowym, w którym zauważone zostały nieprawidłowości było intensywne czyszczenie i nakładanie zmian na ten sam obszar. Przypadek ten skutkuje migotaniem dla klientów i ustępuje w momencie zaprzestania edycji. Rozwiązaniem jest zmiana podejścia do konceptu narzędzia zmywania, podejście resetowaniem pikseli okazuje się nieskuteczne i mniej wydajne niż modyfikacja koloru na bazowy (początkowy kolor płótna).

W przypadku zastosowanych modyfikacji błąd został rozwiązany, spełniając wszystkie scenariusze testowe, które zostały zdefiniowane dla aplikacji.

9. Możliwości rozwoju aplikacji

W przyszłości system może zostać rozbudowany o:

- **Cofanie zmian / historia** - przechowywane w postaci listy *CanvasChange* lub pojedynczej ostatniej aktualizacji u każdego użytkownika indywidualnie, możliwość cofnięcia całej historii lub pojedynczej zmiany
- **Wgrywanie początkowego stanu tablicy w celu edycji** - przetworzenie załadowanego obrazu na piksele, stworzenie listy typu *CanvasChange*, a następnie wywołanie funkcji rysujących na aktualnym płótnie
- **Większy wybór narzędzi do rysowania** - tworzenie kształtów przy pomocy funkcji obsługujących myszkę - zapisując punkt początkowy i końcowy

10. Wnioski

Projekt spełnił wszystkie założenia funkcjonalne zarówno po stronie klienta, jak i serwera. Aplikacja działa stabilnie, a mechanizmy synchronizacji i buforowania zapewniają płynną współpracę między wieloma użytkownikami bez zauważalnych opóźnień. Początkowe obawy dotyczące możliwego spadku wydajności przy większej liczbie klientów okazały się nieuzasadnione. Realizacja projektu pozwoliła również na praktyczne zrozumienie kluczowych zagadnień, takich jak blokowanie dostępu współbieżnego, przetwarzanie wielowątkowe oraz projektowanie rozproszonej komunikacji sieciowej. Całość stanowi dobrze przemyślaną i efektywną implementację wspólnej przestrzeni roboczej online.